Understanding Python Closures

A Series of 8 Progressive Programming Challenges

What you will explore

You will explore scoping, environment capture, state management, and real-world patterns like rate limiters, memoizers, and custom timers.

Exercise 1: Basic Greeting Closure (Scope Warm-up)

Objective: Build a closure factory make_greeter(prefix) that generates customized greeting functions.

Core Concept: Lexical Scope and Enclosing environment. In Python, an inner nested function natively retains references to variables from its parent function, even after the parent function has completed its execution.

Code Template and Verification:

```
# Starter Code Template
def make_greeter(prefix):
    def greeter(name):
        # TODO: Return the formatted greeting using prefix and name
        pass
    return greeter

# Verification Code
welcome_greet = make_greeter("Welcome")
formal_greet = make_greeter("Good morning")
print(welcome_greet("Alice")) # Expected: "Welcome, Alice!"
print(formal_greet("Bob")) # Expected: "Good morning, Bob!"
```

Expected Shell Output:

```
Welcome, Alice!
Good morning, Bob!
```

Exercise 2: Stateful Multiplier Factory (Capturing Environment)

Objective: Implement a multiplier factory make_multiplier(factor) that creates functions multiplying their input by a closed-over factor.

Core Concept: Environment Capturing. This exercise demonstrates how variables defined in an enclosing scope are permanently packed with the returned function.

Code Template and Verification:

```
# Starter Code Template
def make_multiplier(factor):
    def multiplier(number):
        # TODO: Capture 'factor' to calculate the output
        pass
    return multiplier

# Verification Code
double = make_multiplier(2)
triple = make_multiplier(3)
print(double(15)) # Expected: 30
print(triple(15)) # Expected: 45
```

Expected Shell Output:

```
30
45
```

Exercise 3: Simple Event Logger (Tracking State)

Objective: Build a stateful logger make_logger() that preserves an execution history list in memory without using global variables.

Core Concept: Mutable Enclosing Variables. Because lists are mutable, we can append entries directly without declaring special nonlocal bindings.

Code Template and Verification:

```
# Starter Code Template
def make_logger():
    history = []
    def log(message):
        # TODO: Append the message to history and return the full list
        pass
    return log

# Verification Code
logger = make_logger()
print(logger("User logged in")) # Expected: ['User logged in']
print(logger("Query executed")) # Expected: ['User logged in', 'Query executed']
```

Expected Shell Output:

```
['User logged in']
['User logged in', 'Query executed']
```

Exercise 4: Stateful Counter with nonlocal (Modifying State)

Objective: Build a function make_counter() that implements an isolated stateful counter incremented on each call.

Core Concept: State Modification and 'nonlocal'. Integers are immutable in Python. Attempting to modify them inside an inner function will trigger an UnboundLocalError unless we explicitly declare the variable as 'nonlocal' to bind it to the outer enclosing scope.

Code Template and Verification:

```
# Starter Code Template
def make_counter():
    count = 0
    def counter():
        # TODO: Increment and return the outer count variable
        pass
    return counter

# Verification Code
counter_a = make_counter()
print(counter_a()) # Expected: 1
print(counter_a()) # Expected: 2
```

Expected Shell Output:

```
1
2
```

Exercise 5: API Rate Limiter (Real-world Utility)

Objective: Write a rate limiter closure make_rate_limiter(max_calls) that restricts function calls, printing a warning and refusing execution once the threshold is crossed.

Core Concept: Execution Gateways. Closures can wrap functional behaviors to inspect execution thresholds dynamically.

Code Template and Verification:

```
# Starter Code Template
def make_rate_limiter(max_calls):
    calls_made = 0
    def rate_limiter(func, *args, **kwargs):
        # TODO: Track count, block execution if calls_made >= max_calls
        pass
    return rate_limiter

# Verification Code
def get_data(): return "Data"
limiter = make_rate_limiter(2)
print(limiter(get_data)) # Expected: "Data"
print(limiter(get_data)) # Expected: "Data"
print(limiter(get_data)) # Expected: WARNING printed and None returned
```

Expected Shell Output:

```
Data
Data
WARNING: Rate limit exceeded! Request blocked.
None
```

Exercise 6: Memoization Cache (Advanced Closures)

Objective: Build a caching closure make_cached(func) that wraps an expensive function and stores previously computed arguments in a dictionary cache.

Core Concept: Memoization. Using closure dictionary bindings to build isolated cache pools.

Code Template and Verification:

```
# Starter Code Template
def make_cached(func):
    cache = {}
    def cached_wrapper(x):
        # TODO: Check cache, calculate if missing, return cached result
        pass
    return cached_wrapper

# Verification Code
def slow_square(x): return x * x
memo_square = make_cached(slow_square)
print(memo_square(4)) # Expected: Cache Miss and 16
print(memo_square(4)) # Expected: Cache Hit and 16
```

Expected Shell Output:

```
[CACHE MISS] Calculating result for 4...
16
[CACHE HIT] Retrieving cached result for 4...
16
```

Exercise 7: Execution Timer (Prelude to Decorators)

Objective: Build a closure make_timer(func) that intercepts execution to measure and log the timing of a target function in seconds.

Core Concept: Functional Wrappers. Closures can wrap targets using *args and **kwargs to maintain perfect parameter transparency.

Code Template and Verification:

```
# Starter Code Template
import time
def make_timer(func):
    def timer_wrapper(*args, **kwargs):
        # TODO: Record start time, execute func, record end time,
        # print the duration, and return the function result.
        pass
    return timer_wrapper

# Verification Code
def slow_sum(n): return sum(range(n))
timed_sum = make_timer(slow_sum)
timed_sum(1000000)
```

Expected Shell Output:

```
[slow_sum] Executed in X.XXXXXX seconds.
```

Exercise 8: Secure API Token Wrapper (Enclosing Private Properties)

Objective: Create a client closure create_api_client(auth_token) that encloses an authentication token, allowing requests to be sent securely without exposing the raw token attribute.

Core Concept: Information Hiding. Unlike class attributes, closed-over variables are completely inaccessible and invisible from outer namespace lookups, implementing perfect encapsulation.

Code Template and Verification:

```
# Starter Code Template
def create_api_client(auth_token):
    def client(endpoint):
        # TODO: Return mock response dictionary using 'auth_token'
        pass
    return client

# Verification Code
api = create_api_client("secret_token")
print(api("/data")) # Runs request securely
try:
    print(api.auth_token)
except AttributeError:
    print("Inaccessible: token is safe inside closure!")
```

Expected Shell Output:

```
Sending authenticated request to /data...
{...}
Inaccessible: token is safe inside closure!
```